

agentincident.com

Founding Thesis & Product Specification

The failure forensics layer for autonomous agents.

DOCUMENT	Founding Report v1.0
DATE	February 10, 2026
STATUS	COMMITTED
CLASSIFICATION	Internal / Investor-ready

We stop trying to make agents safe. We start making them insurable.

Contents

1	The Problem	Why agents can't be deployed at scale
2	The Thesis	The one-sentence compression
3	The Causal Chain	Nine load-bearing links from capability to deployment
4	Competing Frames	What others think comes next, and why they're wrong
5	The Architecture	Verified rails, market allocation, liability boundaries
6	The Bootstrapping Problem	Why the NTSB for agents must come first
7	The Product	agentincident.com — what we build and in what order
8	The Schema	AgentIncident v0.1 — the five-field incident object
9	The Wedge	Postmortem generator — the smallest sellable atom
10	Kill Vectors	Four ways this dies, and the solves
11	Falsification Tests	How we know if we're right or wrong
12	The Ask	What we need, what we're building, timeline

1. The Problem

AI agents can now decide, act, and operate autonomously. They can write code, process refunds, manage infrastructure, send emails, and deploy to production. The capability gap is closing fast.

But almost none of them are deployed at scale in high-stakes environments. The reason is not intelligence. It is not cost. It is not speed.

The reason is that nobody can answer: “What happens when it goes wrong?”

When a human employee causes a \$47,000 error, there is a clear chain of accountability: the employee, their manager, the policy they violated, the insurance that covers it, the legal framework that adjudicates it. When an AI agent causes the same error, there is nothing. No incident format. No fault taxonomy. No loss data. No insurance product. No liability boundary.

This is not a technical problem. It is an **institutional** problem. The gap between “agents that can act” and “agents that can be deployed” is an institutional gap.

agentincident.com closes this gap.

2. The Thesis

The gap between “agents that can act” and “agents that can be deployed” is an institutional gap, not a capability gap. It will be closed by whoever builds the failure observability layer that makes autonomous decisions priceable.

The supporting logic:

- If an agent cannot be priced, it cannot be insured.
- If it cannot be insured, it cannot be deployed at scale.
- If it cannot be deployed, the capability is stranded.

Pricing requires loss data. Loss data requires standardized failure reporting. Standardized failure reporting requires an incident schema, a recorder, and a corpus. That is what we build.

3. The Causal Chain

The path from “agents exist” to “agents deployed everywhere” is a nine-link dependency chain. Each link is load-bearing. Remove any one and the system fails.

1	You need artificial consequences
2	Artificial consequences require failure classification
3	Failure classification requires standardized observability
4	Observability enables actuarial pricing
5	Pricing enables insurance
6	Insurance enables liability boundaries
7	Liability boundaries enable market deployment
8	Market deployment enables bounded autonomy at scale

The industry is well-resourced on links 0–2 (agent capability) and has begun thinking about links 7–8 (deployment infrastructure). **Links 3–6 are completely unbuilt.** That is our territory.

4. Competing Frames

Several narratives compete for “what comes after agentic engineering.” We evaluated each and explain why they fail or are incomplete.

Autonomous Software Companies

Agents that build, deploy, market, support, and invoice. The “AI startup in a box” dream. **Fails because:** real-world deployment has adversarial edge cases, compliance requirements, trust deficits, and political complexity that pure autonomy cannot navigate.

Self-Healing Systems

Agents that monitor and repair production systems continuously. **Closer, but incomplete:** healing without goal alignment becomes chaotic. Self-repair without accountability is just automated chaos at machine speed.

AI Alignment (Internal Safety)

Making the agent “good” through RLHF, constitutional AI, or evals. **Necessary but insufficient:** you cannot formally verify a neural network the way you verify a protocol. LLMs are stochastic, high-dimensional, and partially non-interpretable. The only thing you can prove is what they are allowed to touch, how far they can go, and what happens when they misbehave.

Agentic Org Charts

CEO agent, sales agent, ops agent, compliance agent. **A trap:** mapping human organizational structure onto agents imports human failure modes at machine speed. The winning architecture is not a company. It is a **market** — agents bidding on tasks, priced by reliability, with automatic reallocation when someone underperforms.

Our frame: Institutional Infrastructure for Bounded Autonomy.

We don’t make agents safe. We make them **accountable**. We verify the cage, not the animal. This is the same move civilization always makes when it encounters powerful actors it cannot fully understand: it builds courts, contracts, audits, and insurance.

5. The Architecture

The core mechanism has three layers:

Layer	Function	Analogy
Verified Rails	Formally verified infrastructure that bounds what agents can do. API limits, budget caps, action grammars, post-condition checks.	Exchange circuit breakers, margin requirements

Market Allocation	Agents bid for tasks, priced by reliability, with slashing for failures and escrow for rewards. Automatic reassignment.	Financial markets coordinating untrusted actors
Liability Boundary	Human veto at the liability edge. Insurance products. Actuarial pricing. Legal accountability mapping.	Product liability law, professional insurance

The critical insight: **we are not verifying the agent. We are verifying the institution the agent is forced to operate inside.** The agent remains a black box. The infrastructure is the provably safe cage around a fundamentally unprovable actor.

This means formal methods and economics converge: formal constraints handle worst-case safety (tail risk), economic alignment handles average-case efficiency (expected value). Game theory without formal methods is a pitch deck. Formal methods without game theory is an academic paper. The winning stack is the fusion.

6. The Bootstrapping Problem

Everything described above requires **loss data**. Insurance companies price policies from claims history, not first principles. But we are at the beginning of autonomous agent deployment. There is no claims history. No standardized failure taxonomy. No agreed ontology of what counts as an agent malfunction vs. a specification error vs. an adversarial input vs. user negligence.

This creates a chicken-and-egg problem:

- Agents can't deploy at scale without insurance
- Insurance can't price accurately without deployment-scale loss data
- Adoption stalls in the gap

The historical precedent: **aviation safety**. It did not come from better pilots or better planes. It came from the NTSB mandating standardized incident reporting, which created the data layer that made actuarial pricing possible, which made aviation insurance viable, which made commercial aviation economically feasible at scale.

The equivalent sequence for agents:

1	→	Standardized incident reporting	THE SCHEMA
2	→	Failure taxonomy convergence	THE RECORDER
3	→	Actuarial data accumulation	THE CORPUS
4	→	Insurance products emerge	THE MARKET
5	→	Mass deployment becomes viable	THE ERA

The entity that solves the bootstrapping problem wins the era.

That entity is agentincident.com. We build step 1 — the schema and the recorder — which bootstraps step 2 and 3. Steps 4 and 5 follow from accumulated data.

7. The Product

agentincident.com generates the incident report your CTO, legal team, and insurer will demand after your agent causes damage.

Four layers, shipped in sequence:

Layer 0: Open Spec

THE LANGUAGE

BUILD FIRST

Five-field incident schema. Open-source. Vendor-neutral. The JSON object that bootstraps the institutional stack. Fault taxonomy, deterministic replay format, severity scoring tied to economic blast radius, loss amount as actuarial anchor.

Layer 1: SDKs & Recorder

DISTRIBUTION

BUILD SECOND

Drop-in instrumentation for every major agent framework. Python SDK, TypeScript SDK, OpenTelemetry exporter. Default-on in one major framework. The Stripe move: make the right thing the default. The schema spreads through usage, not evangelism.

Layer 2: Platform & Corpus

THE BUSINESS

FIRST REVENUE

Hosted review UI + incident aggregation + anonymized sharing. Failure distributions, agent reliability scores, industry benchmarks. Teams use it privately first, then opt in to the corpus in exchange for comparative data. The cross-vendor loss database is the moat.

Layer 3: Actuarial API

THE ENDGAME

THE MOAT

Data feeds for insurers to price agent liability policies. Failure distribution modeling, expected loss calculations by agent type, policy language for coverage boundaries. First partnership with a forward-thinking insurtech carrier.

8. The Schema: AgentIncident v0.1

A civilization-grade flywheel starts from a JSON file and a few enums.

One JSON object per incident. Five core fields. Enough to bootstrap an actuarial table.

Field	Type	Purpose
trace	Object[]	Ordered tool calls, inputs, outputs, timestamps, content hashes. Deterministic replay format with tamper-evidence.
constraints	Object[]	Active policy set at time of incident. Which policy was evaluated, pass/fail, breach delta. The compliance anchor.
fault_class	Enum	AGENT_ERROR SPEC_AMBIGUITY TOOL_FAILURE ADVERSARIAL_INPUT USER_FAULT UNKNOWN. Foundation for fault statistics.
impact_score	Int 1–5	Economic blast radius: money lost, irreversible actions, reputational harm, downtime. Severity tied to economics.
loss_amount_usd	Float (nullable)	Actual dollar loss. The actuarial anchor. Insurers price expected loss, not abstract severity. Supplemented by loss_estimate_usd with breakdown.

Extended fields for the closed-loop defense artifact:

- **classification** — fault_class + subclass + fault_vector + confidence + classifier + status + evidence[] + counterfactuals[] + review trail
- **liability** — primary responsibility + contributing factors + recommended policy owner
- **response** — remediation actions + constraint updates + verification results + time_to_detect / contain / remediate

The closed loop — incident → classification → loss → response → constraint_update → verification — is what transforms a confession into a **defense artifact**. Documentation of failure *plus remediation* is legally stronger than undocumented ignorance. This is the SOC 2 / ISO 27001 playbook applied to agent operations.

9. The Wedge: Postmortem Generator

The smallest sellable atom. Ship first. Charge tomorrow.

Paste your agent logs. Get a professional incident report in 2 minutes.

Day-one product

A web app. One page. Paste trace JSON or upload a log file. Describe what went wrong in 1–2 sentences. Optionally enter the dollar amount involved. Hit generate. Get a structured incident report in the canonical 11-section format: executive summary, impact, timeline, root cause, fault classification, contributing factors, immediate remediation, preventive actions, policy changes, verification plan, trace appendix.

Output: clean markdown (copy button), downloadable PDF, Slack export. Every report includes the agentincident.com footer — every report posted to a Slack channel or attached to a Jira ticket becomes a distribution vehicle.

Why this sells

Right now, when an agent screws up, someone spends 1–3 hours manually reconstructing what happened and writing it up in a Google Doc. agentincident.com reduces this to 2 minutes and produces an artifact that looks like grown-up infrastructure. It is a **time-savings sale**, not a vision sale. Nobody needs to believe in the insurance thesis.

Pricing

Tier	Price	Reports/mo
Pay-per-report	\$19 one-time	1
Solo	\$29/mo	20
Team	\$99/mo	200
Company	\$299/mo	2,000
Enterprise	Custom	Unlimited + retention controls

Beachhead customers

Primary: Fintech ops teams running AI-assisted refund/chargeback workflows. They have the pain, the regulatory pressure, near-misses weekly, and dollar amounts on every action. The `loss_amount_usd` field is never null for them.

Secondary: Dev teams using AI to open PRs and deploy to staging/production. Lower dollar amounts, extremely high near-miss frequency. Fills the corpus fast.

Tertiary: Healthcare AI triage and customer support with tool actions. Higher stakes, slower sales cycle, permanent retention.

10. Kill Vectors

Four ways this dies. Each has a solve. If any solve fails, the company dies.

Kill Vector 1: Demand timing

Threat: Full-autonomy incidents are rare in 2026. Agents aren't deployed with enough blast radius yet.

Solve: near-miss reporting. The ASRS model. Target "what almost broke" instead of "what broke." Near-misses are 100x more frequent, carry zero political cost, and produce the same distributional data. The day-one product works for guardrail-blocked events, not just incidents. This is the bridge that keeps the business alive until the world catches up.

Kill Vector 2: Litigation discovery

Threat: Faultline reports create a structured paper trail of failures. Discoverable in litigation. Companies that document failures become more legally exposed than those that don't.

Solve: defense artifact framing. An incident report without remediation is a confession. An incident report with remediation is evidence of control. The closed-loop format — incident → classification → response → constraint update → verification — proves due diligence. Default-ephemeral storage. Explicit redaction tools. Classification status is always "advisory unless human-confirmed."

Kill Vector 3: Incumbent preemption

Threat: Datadog, Sentry, or AWS bolts on fault classification. They already own the customer relationship.

Solve: ontology advantage + corpus moat. Incumbents think in logs/errors/latency. We think in liability objects. That's a different ontology. The cross-vendor, anonymized loss corpus is the moat — no single vendor has visibility across all frameworks and industries. Win the corpus race before incumbents realize the category exists.

Kill Vector 4: Classification trust

Threat: The fault_class label gets used in disputes. Whoever gets blamed challenges the classification. LLM-based classification creates a recursive trust problem.

Solve: explainable, contestable adjudication. Classification is an object, not a value: includes confidence, evidence, counterfactuals, and review trail. The LLM is the paralegal, not the judge. Human signs off. Status field: draft → confirmed → disputed → revised. Fully auditable.

11. Falsification Tests

A thesis that cannot be falsified is not a thesis.

The thesis is wrong if:

- By 2030, companies still hire “agent engineers” mainly to build workflows and bots, with no convergence on institutional accountability infrastructure.
- The world converges on closed proprietary incident reporting with no shared taxonomy.
- Agent capabilities plateau such that autonomy never exceeds what human review can govern.

The thesis is confirmed if we see:

- Standardized agent incident IDs and public reporting norms
- Policy/spec languages for LLM agents becoming standard tooling
- Agent insurance becoming a real product category with actuarial pricing
- Autonomous systems being sold with warranties
- Job titles like Autonomy Safety Engineer, AI Governance Architect, Agent Reliability Engineer

The single sharpest signal: when Lloyd's of London underwrites an autonomous agent's decisions, the era has arrived. The actuarial tables they build to price those policies will **be** the formal specification of acceptable agent behavior, expressed in dollars.

12. The Ask

Phase	Timeline	Deliverable	Milestone
Weekend MVP	Week 1	Paste-and-generate web app + Stripe checkout	First paid report
Framework SDK	Month 1–2	Python SDK + one agent framework integration	10 teams using it
Review UI	Month 3–4	Hosted review queue + export + Slack	First recurring revenue
Corpus	Month 5–8	Anonymized sharing + industry benchmarks	First benchmark report published
Insurer API	Month 9–12	Data feed for insurtech partner	First policy written against our data

Every time humanity encounters a powerful new actor it cannot fully understand or control — corporations, derivatives, nuclear energy, autonomous vehicles — it follows the same playbook: observe failures, classify them, price the risk, insure it, regulate it, scale it.

We mapped that playbook onto AI agents and found that **nobody has started step one**.

The most transformative technology in a generation is bottlenecked not on compute, not on data, not on architecture — but on **bookkeeping**. On someone building the boring, standardized, unglamorous incident report form that makes everything else possible.

The next era is not better autonomy. It is **accounting for autonomy**.

We stop trying to make agents safe. We start making them insurable.

That is the moment the technology stops being a demo and becomes infrastructure.

agentincident.com

The failure forensics layer for autonomous agents.